Stable-Baselines3 DQN

Blocks with Target 2x4, 4x4, 5x5, …. 10x10

- Which Algorithm?
  - DQN working better than PPO in early experiments
  - DQN is a popular choice for this type of problem (e.g.Rubik's Cube)
- Basics of DQN
  - **Q-Network (Policy):** A neural network (often MlpPolicy or CnnPolicy) that takes the state as input and outputs Q-values for each possible discrete action.
  - **Target Network:** A slowly updated, identical copy of the Q-network used to compute the target Q-value, which helps stabilize training by preventing the network from chasing its own tail.
  - **Replay Buffer:** Stores past experiences to break the correlation between consecutive samples, allowing the agent to learn from a diverse, random batch of past data.
  - **Epsilon-Greedy Exploration:** The agent balances exploration and exploitation by choosing a random action with probability (epsilon) or the best-predicted action

DQN

1. **Interaction & Collection:** The agent interacts with the environment, taking actions and storing transitions in the ReplayBuffer.
2. **Warm-up:** For a specified number of steps (learning_starts), the agent acts randomly to fill the buffer before learning begins.
3. **Sampling:** After the warm-up, the algorithm samples a random mini-batch of experiences from the replay buffer.
4. **Target Calculation:** The target network computes the target Q-value

$$y = r + \gamma \max_{a'} Q_{target}(s', a')$$

5. **Loss Calculation & Update**: The main Q-network computes the current Q(s,a) and updates its weights by minimizing the Mean Squared Error between Q(s,a) and y
6. **Target Network Update**: Every target_update_interval steps, the main network weights are copied to the target network

DQN action space must be Discrete

- The Python-based environment, blocks with target, uses MultiDiscrete actions
- Q: How can we train DQN on this environment?
- A: wrap it!

DiscreteActionWrapper

```
class DiscreteActionWrapper(gym.ActionWrapper):
    def __init__(self, env):
        super().__init__(env)
        # Assume env.action_space is MultiDiscrete([2, 3])
        self.dims = env.action_space.nvec
        self.action_space = gym.spaces.Discrete(np.prod(self.dims))
    def action(self, action):
        # Convert single integer back to tuple for the inner env
        return np.unravel_index(action, self.dims)
```

Applying wrappers to an environment

```
# Define a function that applies all your wrappers
def make_custom_env():
    import gymnasium as gym
    # using 4 blocks and 4 positions right now
    env = gym.make("blocks_env/BlocksTargetPython-v0", num_blocks=4, num_positions=4)
    # Manually pass kwargs to each wrapper here
    env = TimeLimit(env, max_episode_steps=200)
    env = DiscreteActionWrapper(env)
    return env
# Use the function as the env_id, and create 4 parallel copies
env = make_vec_env(make_custom_env, n_envs=4)
```

Logs and trained model storage

```
# Create directories for models and logs

models_dir = "models/dqn"

logs_dir = "logs/dqn"

os.makedirs(models_dir, exist_ok=True)

os.makedirs(logs_dir, exist_ok=True)
```

DQN hyperparameters

model = DQN("MultiInputPolicy", env, learning_starts=100, device=”cuda", batch_size=512, verbose=1, tensorboard_log=logs_dir)

https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html

MultiInputPolicy: observations are a dictionary with current and target

env: our wrapped environment

learning_starts: number of random actions before learning starts

device=“cuda”: use CUDA GPU (would be “mps” on a Mac), or “auto”, or “cpu”

batch_size=512: batch size for update

tensorboard_log=logs_dir: log training progress to the specified directory for viewing with tensorboard

Tensorboard

- Tensorboard can plot information stored in the appropriate format
- Can view graphs of training progress, comparing several runs

Training the model

```
# Train for 1,000,000 timesteps with progress reports every 10,000 steps
    callback = ProgressCallback(check_freq=10000)
    model.learn(total_timesteps=1000000, log_interval=1, callback=callback)
    model.save(f"{models_dir}/dqn_blocks_world")
```

Logging

With **tensorboard_log=logs_dir**, SB3 initializes a global logger that handles multiple output formats simultaneously: **terminal (stdout)** and **TensorBoard binary files**

ProgressCallback has access to this same logger via self.logger. Any custom metrics recorded in the callback using self.logger.record("key", value) will automatically appear in TensorBoard graphs.

log_interval=1: For DQN, this tells SB3 to write a data point to TensorBoard every **1 episode**. This includes standard metrics like rollout/ep_rew_mean and train/loss.

check_freq=10000: callback only triggers its logic every **10,000 timesteps**.

The Result: high-resolution data in TensorBoard (every episode), while terminal/callback reports will only update in massive 10,000-step jumps

Running trained models

```
model = DQN.load(f"{models_dir}/dqn_blocks_world", env)
obs = env.reset()
for _ in range(1000):
    action, _states = model.predict(obs, deterministic=True)
    #obs, reward, terminated, truncated, info = env.step(action)
    obs, reward, terminated, info = env.step(action)
```